Notebooks

Calepin turns ordinary Typst documents into computational notebooks. You write prose, equations, figures, tables, and page layout in Typst, then place executable code directly in the same `.typ` file. When you run `calepin compile notebook.typ`, *Calepin* scans the document for chunks, runs them, stores their results, and asks Typst to render the final document with those results inserted in place.

During compilation, *Calepin* creates a hidden `.calepin` directory beside your document. That directory is an implementation detail and can be regenerated, but it contains the Typst runtime file that notebooks use while rendering. Every *Calepin* notebook should import those Typst functions at the top of the document:

```
#import "/.calepin/calepin.typ" as calepin
```

Use that import, not Typst Universe, for now. You may see a `calepin` package on Typst Universe, but it is currently only a placeholder and should not be used while *Calepin* is evolving quickly.

Standard Typst

A *Calepin* notebook is still a standard Typst document. Headings, paragraphs, lists, math, links, functions, variables, and layout rules are normal Typst. The notebook behavior comes from a small set of imported helper functions and from fenced code blocks that *Calepin* sees before Typst renders the document.

That means a notebook can begin like any other Typst file:

```
#import "/.calepin/calepin.typ" as calepin

#set document(title: [My analysis])

= Introduction

This is regular Typst prose. The expression $pi r^2$ is regular Typst math.
```

Code chunks

The simplest executable chunk is an ordinary fenced code block named after its language. *Calepin* runs the block and places the captured output below the source:

````
```python
print(40 + 2)
```
````

```python
print(40 + 2)
```

```
42
```

Languages run in persistent sessions, so variables defined in one chunk are available in later chunks:

````
```python
x = 41
```

```python
print(x + 1)
```
````

```python
x = 41
```

```python
print(x + 1)
```

```
42
```

Set document-wide defaults with `#calepin.setup(...)`. For example, this page uses `results: "verbatim"` so console output is shown as plain text. Other pages use `results: "render"` when plots, rich values, or Typst output should be rendered more fully.

Inline results

Inline code is for computed values that belong inside a sentence. The common pattern is to create a short alias once, then call it where the result should appear:

```
#let py = calepin.inline.with("python")

The answer is #py[`print(40 + 2)`].
```

The answer is 42.

Full example

Here is a complete starter notebook with comments. Save it as `notebook.typ`, run `calepin compile notebook.typ`, and open the generated output.

````
// Import the Calepin Typst runtime generated in .calepin/.
#import "/.calepin/calepin.typ" as calepin

// This is regular Typst metadata.
#set document(title: [My Calepin notebook])

// Defaults apply to every chunk unless a chunk overrides them.
#calepin.setup(
  echo: true,
  eval: true,
  results: "verbatim",
)

// A short alias keeps inline code readable in prose.
#let py = calepin.inline.with("python")

= My Calepin notebook

This paragraph is ordinary Typst.

// A plain fenced code block is executable when its language is supported.
```python
x = 41
print(x + 1)
```

Variables persist across chunks in the same language session:

```python
print(x + 2)
```

Inline results can appear inside prose. The answer is #py[`print(40 + 2)`].

// Use calepin.chunk(...) when a block needs options.
#calepin.chunk(echo: false, results: "typst")[
```python
print("#strong[This text was produced by Python and rendered by Typst.]")
```
]
````